

TEST FOR ANYLOGISTIX API

The API info is here: <https://anylogistix.help/api/python/readme.html>

PHASE 1 Connect API, get scenarios, select 1, run it and collect the simulated data

Install the dependencies

It assumes that the openapi_client is in the local directory ./openapi-python-client-3.3.1

```
In [3]: import sys
import os
# This is my personal directory
os.chdir(r"C:\Users\Luis\Downloads\TFG\API")
#!{sys.executable} -m pip install "pydantic>=2.10,<3"
!{sys.executable} -m pip install "pydantic<2,>=1.10.5"
!{sys.executable} -m pip install -e ./openapi-python-client-3.3.1
```

```

Requirement already satisfied: pydantic<2,>=1.10.5 in c:\users\luis\downloads\tfg\anaconda\envs\tfg_anylogistix\lib\site-packages (1.10.26)
Requirement already satisfied: typing-extensions>=4.2.0 in c:\users\luis\downloads\tfg\anaconda\envs\tfg_anylogistix\lib\site-packages (from pydantic<2,>=1.10.5) (4.15.0)
Obtaining file:///C:/Users/Luis/Downloads/TFG/API/openapi-python-client-3.3.1
  Installing build dependencies: started
  Installing build dependencies: finished with status 'done'
  Checking if build backend supports build_editable: started
  Checking if build backend supports build_editable: finished with status 'done'
  Getting requirements to build editable: started
  Getting requirements to build editable: finished with status 'done'
  Preparing editable metadata (pyproject.toml): started
  Preparing editable metadata (pyproject.toml): finished with status 'done'
Requirement already satisfied: urllib3<2.1.0,>=1.25.3 in c:\users\luis\downloads\tfg\anaconda\envs\tfg_anylogistix\lib\site-packages (from openapi-client==1.0.0) (2.0.7)
Requirement already satisfied: python-dateutil in c:\users\luis\downloads\tfg\anaconda\envs\tfg_anylogistix\lib\site-packages (from openapi-client==1.0.0) (2.9.0.post0)
Requirement already satisfied: pydantic<2,>=1.10.5 in c:\users\luis\downloads\tfg\anaconda\envs\tfg_anylogistix\lib\site-packages (from openapi-client==1.0.0) (1.10.26)
Requirement already satisfied: aenum in c:\users\luis\downloads\tfg\anaconda\envs\tfg_anylogistix\lib\site-packages (from openapi-client==1.0.0) (3.1.17)
Requirement already satisfied: typing-extensions>=4.2.0 in c:\users\luis\downloads\tfg\anaconda\envs\tfg_anylogistix\lib\site-packages (from pydantic<2,>=1.10.5->openapi-client==1.0.0) (4.15.0)
Requirement already satisfied: six>=1.5 in c:\users\luis\downloads\tfg\anaconda\envs\tfg_anylogistix\lib\site-packages (from python-dateutil->openapi-client==1.0.0) (1.17.0)
Building wheels for collected packages: openapi-client
  Building editable for openapi-client (pyproject.toml): started
  Building editable for openapi-client (pyproject.toml): finished with status 'done'
  Created wheel for openapi-client: filename=openapi_client-1.0.0-0.editable-py3-none-any.whl size=3042 sha256=b3059e5926115896a4ed12a11556076e447d928d34fe080e3b7515423230f201
  Stored in directory: C:\Users\Luis\AppData\Local\Temp\pip-ephem-wheel-cache-6vn1tqk8\wheels\96\67\f9\218cef519d1de93e10ecbd2f12fb5f4d1d0e81b1c1d5296ba2
Successfully built openapi-client
Installing collected packages: openapi-client
  Attempting uninstall: openapi-client
    Found existing installation: openapi-client 1.0.0
    Uninstalling openapi-client-1.0.0:
      Successfully uninstalled openapi-client-1.0.0
Successfully installed openapi-client-1.0.0

```

```

In [4]: import openapi_client
        from openapi_client.rest import ApiException
        from pprint import pprint
        import urllib3

        import shutil #For manipulating excel files
        import re # For cleaning strings

```

```

In [5]: # SERVER_IP="192.168.64.159"
        # SERVER_IP="192.168.67.110"
        SERVER_IP="alxserver.aut.uah.es"
        SERVER_URL=f"https://{SERVER_IP}:443/api/v1"

```

```
# This is my personal APY KEY
API_KEY="c184f1ab-9f13-484c-a1c1-3d543502da6e"

SERVER_URL
```

Out[5]: 'https://alxserver.aut.uah.es:443/api/v1'

PART I: CHECK CONNECTIVIY AND OPERATIONAL STATUS

```
In [6]: # Defining the host is optional and defaults to https://server_address:port/api/
# See configuration.py for a list of all supported configuration parameters.
configuration = openapi_client.Configuration(
    host = SERVER_URL
)
# The client must configure the authentication and authorization parameters
# in accordance with the API server security policy.
# Examples for each auth method are provided below, use the example that
# satisfies your auth use case.
# Configure API key authorization: ApiKey
configuration.api_key['ApiKey'] = API_KEY

# ALVARO: SUPER IMPORTANTE PARA PODER TRABAJAR CON CERTIFICADOS AUTO-FIRMADOS
configuration.verify_ssl = False
urllib3.disable_warnings(urllib3.exceptions.InsecureRequestWarning)
with openapi_client.ApiClient(configuration) as api_client:
    # Create an instance of the API class
    api_instance = openapi_client.OpenApiApi(api_client)
```

Obtain current user data

```
In [7]: try:
    # Gets current user data.
    api_response = api_instance.get_current_user()
    print("The response of OpenApiApi->get_current_user:\n")
    pprint(api_response)
except Exception as e:
    print("Exception when calling OpenApiApi->get_current_user: %s\n" % e)
```

The response of OpenApiApi->get_current_user:

```
ApiUserData(email='adrian.encabo@edu.uah.es', first_name='adrian.encabo@edu.uah.e
s', id=403, last_name='', username='adrian.encabo@edu.uah.es')
```

PART II: OBTAIN PROJECTS, SCENARIOS, EXECUTIONS

Obtain the list of currently available projects

```
In [8]: try:
    # Returns a list of projects that the user has access to.
    api_response = api_instance.get_projects()
    print("The response of OpenApiApi->get_projects:\n")
```

```
pprint(api_response)
except Exception as e:
    print("Exception when calling OpenApiApi->get_projects: %s\n" % e)
```

The response of OpenApiApi->get_projects:

```
[ApiProjectResponse(accessible=True, creation_date='2026-03-10T16:19:55.145394Z',
current_user_id=403, deleted=False, id=79020, name='TFG_ADRIAN_ENCABO', owner_user_id=3)]
```

Open a project and get the list of scenarios available

```
In [9]: # My personal Project defined
MY_PROJECT_NAME="TFG_ADRIAN_ENCABO"
the_project_id=0
try:
    full_match = True # bool | Whether to match project name by complete match.
    # Opens project with name projectName.
    the_project = api_instance.find_and_open_project_by_name(full_match, MY_PROJECT_NAME)
    print("The response of OpenApiApi->find_and_open_project_by_name:\n")
    pprint(the_project)
except Exception as e:
    print("Exception when calling OpenApiApi->find_and_open_project_by_name: %s\n" % e)
try:
    # Returns a List of scenarios for the project with identifier project_id.
    the_scenarios = api_instance.get_scenarios(the_project.id)
    print("The response of OpenApiApi->get_scenarios:\n")
    pprint(the_scenarios)
except Exception as e:
    print("Exception when calling OpenApiApi->get_scenarios: %s\n" % e)
```

The response of OpenApiApi->find_and_open_project_by_name:

```
ApiProjectResponse(accessible=True, creation_date='2026-03-10T16:19:55.145394Z',
current_user_id=403, deleted=False, id=79020, name='TFG_ADRIAN_ENCABO', owner_user_id=3)
```

The response of OpenApiApi->get_scenarios:

```
[ApiScenarioData(id=82840, name='Budget Comparison (Estimated Demand)', type='SIM'),
 ApiScenarioData(id=87083, name='P3.3. GFA 1. Results 2_Different Locations', type='SIM'),
 ApiScenarioData(id=84191, name='Budget Comparison (Estimated Demand) 2', type='SIM'),
 ApiScenarioData(id=85538, name='Budget Comparison (Estimated Demand) 2', type='SIM'),
 ApiScenarioData(id=86671, name='P4.2.2 NO Distribution Network inside 4 Walls Models Capacity Restriction', type='NO'),
 ApiScenarioData(id=88185, name='P4.2.2 NO Distribution Network inside 4 Walls Models Capacity Restriction 2', type='NO'),
 ApiScenarioData(id=89212, name='P4.2.2 NO Distribution Network inside 4 Walls Models Capacity Restriction sim', type='SIM'),
 ApiScenarioData(id=92735, name='P4.2.6 SIM Distribution Network-4 Walls_New Inventory+New Transportation', type='SIM'),
 ApiScenarioData(id=95044, name='null', type='SIM'),
 ApiScenarioData(id=176431, name='Cold Chain (Risk Analysis)', type='SIM'),
 ApiScenarioData(id=177767, name='US Distribution Network', type='NO'),
 ApiScenarioData(id=178297, name='Lumber Production', type='NO'),
 ApiScenarioData(id=179567, name='Global Network Examination', type='SIM'),
 ApiScenarioData(id=181707, name='P3.1._Green_Field_Analysis_GFA_ALX3', type='GFA'),
 ApiScenarioData(id=180669, name='asset-v1_UAH+EUGLOH+EUG2_T2_1_0080+type@asset+block@P4.4_Demand_Variation_LoadingGates_2', type='SIM'),
 ApiScenarioData(id=192031, name='null 6', type='SIM'),
 ApiScenarioData(id=200223, name='P4.2.6 SIM Distribution Network-4 Walls_New Inventory+New Transportation 2', type='SIM'),
 ApiScenarioData(id=197360, name='null 8', type='SIM'),
 ApiScenarioData(id=184791, name='null 3', type='SIM'),
 ApiScenarioData(id=187186, name='null 4', type='SIM'),
 ApiScenarioData(id=182396, name='null 2', type='SIM'),
 ApiScenarioData(id=189602, name='null 5', type='SIM'),
 ApiScenarioData(id=194446, name='null 7', type='SIM'),
 ApiScenarioData(id=213266, name='null 13', type='SIM'),
 ApiScenarioData(id=202944, name='null 9', type='SIM'),
 ApiScenarioData(id=205644, name='null 10', type='SIM'),
 ApiScenarioData(id=222111, name='null 15', type='SIM'),
 ApiScenarioData(id=208189, name='null 11', type='SIM'),
 ApiScenarioData(id=218720, name='null 14', type='SIM'),
 ApiScenarioData(id=210209, name='null 12', type='SIM'),
 ApiScenarioData(id=230388, name='null 17', type='SIM'),
 ApiScenarioData(id=226034, name='null 16', type='SIM'),
 ApiScenarioData(id=234686, name='null 18', type='SIM'),
 ApiScenarioData(id=238984, name='null 19', type='SIM'),
 ApiScenarioData(id=243908, name='null 20', type='SIM'),
 ApiScenarioData(id=249011, name='null 21', type='SIM'),
 ApiScenarioData(id=253735, name='null 22', type='SIM'),
 ApiScenarioData(id=257931, name='null 23', type='SIM'),
 ApiScenarioData(id=263418, name='null 24', type='SIM'),
 ApiScenarioData(id=268694, name='Global Network Examination 2', type='SIM'),
 ApiScenarioData(id=270639, name='Modified_AI_Decision_0', type='SIM')]
```

Select the scenario by name

```
In [10]: #Add the name of the scenario
MY_SCENARIO_NAME="Global Network Examination"

# 1. Select the scenario
the_scenario = next(s for s in the_scenarios if s.name == MY_SCENARIO_NAME)
pprint(the_scenario)
```

```
ApiScenarioData(id=179567, name='Global Network Examination', type='SIM')
```

Get the list of experiment runs of a scenario

```
In [11]: try:
# Returns the results of experiment runs for scenario.
the_runs = api_instance.get_experiment_runs_for_scenario(the_scenario.id)
print("The response of OpenApiApi->get_experiment_runs_for_scenario:\n")
pprint(the_runs)
except Exception as e:
print("Exception when calling OpenApiApi->get_experiment_runs_for_scenario:
```

The response of OpenApiApi->get_experiment_runs_for_scenario:

```
[ApiBasicRun(has_options=False, id=201331, name='Statistics', rc_id=201332, removing=False, scenario_id=179567, type='SIMULATION'),
 ApiBasicRun(has_options=False, id=201578, name='Statistics 2', rc_id=201579, removing=False, scenario_id=179567, type='SIMULATION'),
 ApiBasicRun(has_options=False, id=201796, name='Statistics 3', rc_id=201797, removing=False, scenario_id=179567, type='SIMULATION'),
 ApiBasicRun(has_options=False, id=217384, name='Statistics 4', rc_id=217385, removing=False, scenario_id=179567, type='SIMULATION'),
 ApiBasicRun(has_options=False, id=217581, name='Statistics 5', rc_id=217582, removing=False, scenario_id=179567, type='SIMULATION'),
 ApiBasicRun(has_options=False, id=220765, name='Statistics 6', rc_id=220766, removing=False, scenario_id=179567, type='SIMULATION'),
 ApiBasicRun(has_options=False, id=220963, name='Statistics 7', rc_id=220964, removing=False, scenario_id=179567, type='SIMULATION'),
 ApiBasicRun(has_options=False, id=224700, name='Statistics 8', rc_id=224701, removing=False, scenario_id=179567, type='SIMULATION'),
 ApiBasicRun(has_options=False, id=224898, name='Statistics 9', rc_id=224899, removing=False, scenario_id=179567, type='SIMULATION'),
 ApiBasicRun(has_options=False, id=229051, name='Statistics 10', rc_id=229052, removing=False, scenario_id=179567, type='SIMULATION'),
 ApiBasicRun(has_options=False, id=229248, name='Statistics 11', rc_id=229249, removing=False, scenario_id=179567, type='SIMULATION'),
 ApiBasicRun(has_options=False, id=242371, name='Statistics 12', rc_id=242372, removing=False, scenario_id=179567, type='SIMULATION'),
 ApiBasicRun(has_options=False, id=242569, name='Statistics 13', rc_id=242570, removing=False, scenario_id=179567, type='SIMULATION'),
 ApiBasicRun(has_options=False, id=261628, name='Statistics 14', rc_id=261629, removing=False, scenario_id=179567, type='SIMULATION'),
 ApiBasicRun(has_options=False, id=261826, name='Statistics 15', rc_id=261827, removing=False, scenario_id=179567, type='SIMULATION'),
 ApiBasicRun(has_options=False, id=262035, name='Statistics 16', rc_id=262036, removing=False, scenario_id=179567, type='SIMULATION'),
 ApiBasicRun(has_options=False, id=262233, name='Statistics 17', rc_id=262234, removing=False, scenario_id=179567, type='SIMULATION')]
```

PART III. EXECUTE A SIMULATION

Get the list of RunConfigurations for the experiments of a given Scenario

Each type of experiment has a RunConfiguration. For example:

```
ApiBasicRunConfiguration(animation_model_id=None,
feasible_check_status='NOT_CONDUCTED', id=2439,
is_feasible_check_data_available=False, model_time=None,
name='i18n-Experiment_Simulation_text', progress=None,
project_id=63, routes_progress=None, run_status=None,
scenario_id=1752, speed=None, status='IDLE', step='IDLE',
type='SIMULATION', validation_status='VALID')
```

Types of available experiment runs are:

- SIMULATION
- VARIATION
- COMPARISON
- SAFETY_STOCK
- RISK_ANALYSIS

```
In [12]: MY_RUN_TYPE="SIMULATION"
try:
    # Returns a list of experiments available for a given scenario.
    the_run_configurations= api_instance.get_experiments(the_project.id, the_sce
    print("The response of OpenApiApi->get_experiments:\n")
    # Exception fixed for undefined variable
    pprint(the_run_configurations)
except Exception as e:
    print("Exception when calling OpenApiApi->get_experiments: %s\n" % e)

# 2. Select the adequate RunConfiguration for the scenario
the_RC = next((r for r in the_run_configurations if r.type == MY_RUN_TYPE), None)
pprint(the_RC)
```

The response of OpenApiApi->get_experiments:

```
[ApiBasicRunConfiguration(animation_model_id=None, feasible_check_status='NOT_CONDUCTED', id=180254, is_feasible_check_data_available=False, model_time=None, name='i18n-Experiment_Simulation_text', progress=None, project_id=79020, routes_progress=None, run_status=None, scenario_id=179567, speed=None, status='IDLE', step='IDLE', type='SIMULATION', validation_status='VALID'),
 ApiBasicRunConfiguration(animation_model_id=None, feasible_check_status='NOT_CONDUCTED', id=180429, is_feasible_check_data_available=False, model_time=None, name='i18n-Experiment_VariationExperiment_text', progress=None, project_id=79020, routes_progress=None, run_status=None, scenario_id=179567, speed=None, status='IDLE', step='IDLE', type='VARIATION', validation_status='NOT_CONDUCTED'),
 ApiBasicRunConfiguration(animation_model_id=None, feasible_check_status='NOT_CONDUCTED', id=179575, is_feasible_check_data_available=False, model_time=None, name='i18n-Experiment_ComparisonExperiment_text', progress=None, project_id=79020, routes_progress=None, run_status=None, scenario_id=179567, speed=None, status='IDLE', step='IDLE', type='COMPARISON', validation_status='NOT_CONDUCTED'),
 ApiBasicRunConfiguration(animation_model_id=None, feasible_check_status='NOT_CONDUCTED', id=180115, is_feasible_check_data_available=False, model_time=None, name='i18n-Experiment_SafetyStockEstimation_text', progress=None, project_id=79020, routes_progress=None, run_status=None, scenario_id=179567, speed=None, status='IDLE', step='IDLE', type='SAFETY_STOCK', validation_status='NOT_CONDUCTED'),
 ApiBasicRunConfiguration(animation_model_id=None, feasible_check_status='NOT_CONDUCTED', id=179930, is_feasible_check_data_available=False, model_time=None, name='i18n-Experiment_RiskAnalysisExperiment_text', progress=None, project_id=79020, routes_progress=None, run_status=None, scenario_id=179567, speed=None, status='IDLE', step='IDLE', type='RISK_ANALYSIS', validation_status='NOT_CONDUCTED')]
 ApiBasicRunConfiguration(animation_model_id=None, feasible_check_status='NOT_CONDUCTED', id=180254, is_feasible_check_data_available=False, model_time=None, name='i18n-Experiment_Simulation_text', progress=None, project_id=79020, routes_progress=None, run_status=None, scenario_id=179567, speed=None, status='IDLE', step='IDLE', type='SIMULATION', validation_status='VALID')
```

Run a new experiment over a scenario (synch)

Runs synchronously.

```
In [13]: try:
          # Starts the experiment synchronously.
          the_sync_result = api_instance.run_experiment_synchronously(the_RC.id)
          print("The response of OpenApiApi->run_experiment_synchronously:\n")
          pprint(the_sync_result)
        except Exception as e:
          print("Exception when calling OpenApiApi->run_experiment_synchronously: %s\n"
```

The response of OpenApiApi->run_experiment_synchronously:

```
ApiExperimentResult(experiment_result_id=272060, validation_errors=None, validation_status='OK', validation_warnings=None)
```

Part IIIb. Asynch execution

Run a new experiment over a scenario (asynch)

Runs asynchronously. Periodic status check.


```
In [14]: skip_scenario_warnings = True # bool | Whether to skip scenario warnings. (optional)
try:
    # Starts the experiment asynchronously.
    the_execution = api_instance.run_experiment(the_RC.id, skip_scenario_warning
    print("The response of OpenApiApi->run_experiment:\n")
    pprint(the_execution)
except Exception as e:
    print("Exception when calling OpenApiApi->run_experiment: %s\n" % e)

import time
while True:
    try:
        # Returns the experiment status
        the_status = api_instance.get_experiment_status(the_RC.id)
        # pprint(api_response)

        # Check if execution finished
        if the_status.finished:
            print("Execution finished.")
            break

        # Optional: stop if failed
        # fixed exception of status check
        if the_status.failed:
            print("Execution failed.")
            break

    except Exception as e:
        print("Exception when calling OpenApiApi->get_experiment_status:", e)
    print("Not finished yet. Wait 1sec and check again")
    # wait before next check
    time.sleep(1)
```

The response of OpenApiApi->run_experiment:

```
ApiRunExperiment(animation_model_id=180254, experiment_type_id='SIMULATION', is3d
_animation=False)
Execution finished.
```

Get the experiments results (pages)

You can ask for limited number of results (avoid overflow)

```
In [15]: skip = 0 # int | Number of records to skip. (default to 0)
take = 1000 # int | Number of records to be retrieved. (default to 1000)

try:
    # Returns all the results of the specific experiment run.
    the_run_results = api_instance.get_experiment_run_result(the_RC.id, skip, ta
    print("The response of OpenApiApi->get_experiment_run_result:\n")
    pprint(the_run_results)
except Exception as e:
    print("Exception when calling OpenApiApi->get_experiment_run_result: %s\n" %
```

The response of OpenApiApi->get_experiment_run_result:

```
ApiExperimentResultData(pages=[ApiExperimentResultPageWrapper(charts=[ApiExperimentResultGraphChartWrapper(chart=ApiChartMetadataShort(id=180257, is_group_mode_enabled=False, layout=ApiChartLayoutData(height=1, start_col=0, start_row=0, width=2), name='Available Inventory', rc_id=180254, type='LINE'), class_name='ApiExperimentResultGraphChartWrapper', data=ApiGraphChartData(entity_list=[], id=180257, show_x=True, total_entity_count=0, total_technical_row_count=0, type='LINE')), ApiExperimentResultGraphChartWrapper(chart=ApiChartMetadataShort(id=180258, is_group_mode_enabled=False, layout=ApiChartLayoutData(height=1, start_col=0, start_row=1, width=2), name='Available Inventory Including Backlog', rc_id=180254, type='LINE'), class_name='ApiExperimentResultGraphChartWrapper', data=ApiGraphChartData(entity_list=[], id=180258, show_x=True, total_entity_count=0, total_technical_row_count=0, type='LINE')), ApiExperimentResultGraphChartWrapper(chart=ApiChartMetadataShort(id=180259, is_group_mode_enabled=False, layout=ApiChartLayoutData(height=1, start_col=4, start_row=0, width=2), name='Average Daily Available Inventory', rc_id=180254, type='LINE'), class_name='ApiExperimentResultGraphChartWrapper', data=ApiGraphChartData(entity_list=[], id=180259, show_x=True, total_entity_count=0, total_technical_row_count=0, type='LINE')), ApiExperimentResultGraphChartWrapper(chart=ApiChartMetadataShort(id=180260, is_group_mode_enabled=False, layout=ApiChartLayoutData(height=1, start_col=2, start_row=1, width=2), name='Average Daily On-hand Inventory', rc_id=180254, type='LINE'), class_name='ApiExperimentResultGraphChartWrapper', data=ApiGraphChartData(entity_list=[], id=180260, show_x=True, total_entity_count=0, total_technical_row_count=0, type='LINE')), ApiExperimentResultGraphChartWrapper(chart=ApiChartMetadataShort(id=180261, is_group_mode_enabled=False, layout=ApiChartLayoutData(height=1, start_col=2, start_row=0, width=2), name='On-hand Inventory', rc_id=180254, type='LINE'), class_name='ApiExperimentResultGraphChartWrapper', data=ApiGraphChartData(entity_list=[], id=180261, show_x=True, total_entity_count=0, total_technical_row_count=0, type='LINE'))], page=ApiPage(id='180256', name='Available Inventory')), ApiExperimentResultPageWrapper(charts=[ApiExperimentResultTableChartWrapper(chart=ApiChartMetadataShort(id=180263, is_group_mode_enabled=False, layout=ApiChartLayoutData(height=1, start_col=0, start_row=0, width=2), name='Demand Placed, Fulfillment Received (Accumulative)', rc_id=180254, type='TABLE'), class_name='ApiExperimentResultTableChartWrapper', data=ApiStrictTableChartData(entity_list=[], id=180263, total_entity_count=0, total_technical_row_count=0, type='TABLE')), ApiExperimentResultTableChartWrapper(chart=ApiChartMetadataShort(id=180264, is_group_mode_enabled=False, layout=ApiChartLayoutData(height=1, start_col=0, start_row=1, width=2), name='Demand Placed, Fulfillment Received (Accumulative, Per Object)', rc_id=180254, type='TABLE'), class_name='ApiExperimentResultTableChartWrapper', data=ApiStrictTableChartData(entity_list=[], id=180264, total_entity_count=0, total_technical_row_count=0, type='TABLE')), ApiExperimentResultGraphChartWrapper(chart=ApiChartMetadataShort(id=180265, is_group_mode_enabled=False, layout=ApiChartLayoutData(height=1, start_col=2, start_row=0, width=2), name='Demand Placed, Fulfillment Received (Daily)', rc_id=180254, type='LINE'), class_name='ApiExperimentResultGraphChartWrapper', data=ApiGraphChartData(entity_list=[], id=180265, show_x=True, total_entity_count=0, total_technical_row_count=0, type='LINE')), ApiExperimentResultGraphChartWrapper(chart=ApiChartMetadataShort(id=180266, is_group_mode_enabled=False, layout=ApiChartLayoutData(height=1, start_col=2, start_row=1, width=2), name='Demand Placed, Fulfillment Received (Daily, Per Object)', rc_id=180254, type='LINE'), class_name='ApiExperimentResultGraphChartWrapper', data=ApiGraphChartData(entity_list=[], id=180266, show_x=True, total_entity_count=0, total_technical_row_count=0, type='LINE')), ApiExperimentResultTableChartWrapper(chart=ApiChartMetadataShort(id=180267, is_group_mode_enabled=False, layout=ApiChartLayoutData(height=1, start_col=4, start_row=0, width=2), name='Demand Received, Fulfillment Shipped (Accumulative)', rc_id=180254, type='TABLE'), class_name='ApiExperimentResultTableChartWrapper', data=ApiStrictTableChartData(entity_list=[], id=180267, total_entity_count=0, total_technical_row_count=0, type='TABLE')), ApiExperimentResultGraphChartWrapper(chart=ApiChartMetadataShort(id=180268, is_group_mode_enabled=False, layout=ApiChartLayoutData(height=1, start_col=4, start_row=1, width=2), name='Demand Received, Fulfillment Shipped (Daily, Per Object)', rc_id=180254, type='LINE'), class_name='ApiExperimentResultGraphChartWrapper', data=ApiGraphChartData(entity_list=[], id=180268, show_x=True, total_entity_count=0, total_technical_row_count=0, type='LINE'))], page=ApiPage(id='180256', name='Available Inventory'))]
```

```

ent Shipped (Daily)', rc_id=180254, type='LINE')), class_name='ApiExperimentResult
GraphChartWrapper', data=ApiGraphChartData(entity_list=[], id=180268, show_x=Tru
e, total_entity_count=0, total_technical_row_count=0, type='LINE'))], page=ApiPag
e(id='180262', name='Fulfillment')), ApiExperimentResultPageWrapper(charts=[ApiEx
perimentResultGraphChartWrapper(chart=ApiChartMetadataShort(id=180270, is_group_m
ode_enabled=False, layout=ApiChartLayoutData(height=1, start_col=0, start_row=0,
width=2), name='Lead Time', rc_id=180254, type='LINE')), class_name='ApiExperim
entResultGraphChartWrapper', data=ApiGraphChartData(entity_list=[], id=180270, show_
x=True, total_entity_count=0, total_technical_row_count=0, type='LINE')), ApiExpe
rimentResultTableChartWrapper(chart=ApiChartMetadataShort(id=180271, is_group_mod
e_enabled=False, layout=ApiChartLayoutData(height=1, start_col=4, start_row=0, wi
dth=2), name='Max Lead Time', rc_id=180254, type='TABLE')), class_name='ApiExperim
entResultTableChartWrapper', data=ApiStrictTableChartData(entity_list=[], id=1802
71, total_entity_count=0, total_technical_row_count=0, type='TABLE')), ApiExperim
entResultGraphChartWrapper(chart=ApiChartMetadataShort(id=180272, is_group_mode_e
nabled=False, layout=ApiChartLayoutData(height=1, start_col=2, start_row=0, width
=2), name='Mean Lead Time (Best-Mean-Worst)', rc_id=180254, type='BMW-LINE')), cla
ss_name='ApiExperimentResultGraphChartWrapper', data=ApiGraphChartData(entity_lis
t=[], id=180272, show_x=True, total_entity_count=0, total_technical_row_count=0,
type='BMW-LINE'))], page=ApiPage(id='180269', name='Lead Time')), ApiExperimentRe
sultPageWrapper(charts=[ApiExperimentResultGraphChartWrapper(chart=ApiChartMetada
taShort(id=180274, is_group_mode_enabled=False, layout=ApiChartLayoutData(height=
1, start_col=4, start_row=0, width=2), name='Available Inventory Including Backlo
g', rc_id=180254, type='LINE')), class_name='ApiExperimentResultGraphChartWraphe
r', data=ApiGraphChartData(entity_list=[], id=180274, show_x=True, total_entity_c
ount=0, total_technical_row_count=0, type='LINE')), ApiExperimentResultGraphChart
Wrapper(chart=ApiChartMetadataShort(id=180277, is_group_mode_enabled=False, layou
t=ApiChartLayoutData(height=1, start_col=0, start_row=0, width=2), name='Products
Produced (Units) Factory Shenzhen', rc_id=180254, type='BAR')), class_name='ApiExp
erimentResultGraphChartWrapper', data=ApiGraphChartData(entity_list=[], id=18027
7, show_x=True, total_entity_count=0, total_technical_row_count=0, type='BAR')),
ApiExperimentResultGraphChartWrapper(chart=ApiChartMetadataShort(id=180280, is_gr
oup_mode_enabled=False, layout=ApiChartLayoutData(height=1, start_col=2, start_ro
w=0, width=2), name='Products Produced (Units) Factory Xiamen', rc_id=180254, typ
e='BAR')), class_name='ApiExperimentResultGraphChartWrapper', data=ApiGraphChartDa
ta(entity_list=[], id=180280, show_x=True, total_entity_count=0, total_technical_
row_count=0, type='BAR'))], page=ApiPage(id='180273', name='Production')), ApiExp
erimentResultPageWrapper(charts=[ApiExperimentResultTableChartWrapper(chart=ApiCh
artMetadataShort(id=180284, is_group_mode_enabled=False, layout=ApiChartLayoutDat
a(height=1, start_col=2, start_row=0, width=2), name='Profit and Loss Statement',
rc_id=180254, type='TABLE')), class_name='ApiExperimentResultTableChartWrapper', d
ata=ApiStrictTableChartData(entity_list=[], id=180284, total_entity_count=0, tota
l_technical_row_count=0, type='TABLE')), ApiExperimentResultGraphChartWrapper(cha
rt=ApiChartMetadataShort(id=180285, is_group_mode_enabled=False, layout=ApiChartL
ayoutData(height=1, start_col=4, start_row=0, width=2), name='Profit, Revenue, To
tal Cost', rc_id=180254, type='BAR')), class_name='ApiExperimentResultGraphChartWr
apper', data=ApiGraphChartData(entity_list=[], id=180285, show_x=True, total_enti
ty_count=0, total_technical_row_count=0, type='BAR')), ApiExperimentResultGraphCh
artWrapper(chart=ApiChartMetadataShort(id=180286, is_group_mode_enabled=False, la
yout=ApiChartLayoutData(height=1, start_col=0, start_row=0, width=2), name='Reven
ue, Total Cost', rc_id=180254, type='LINE')), class_name='ApiExperimentResultGraph
ChartWrapper', data=ApiGraphChartData(entity_list=[], id=180286, show_x=True, tot
al_entity_count=0, total_technical_row_count=0, type='LINE'))], page=ApiPage(id
='180283', name='Profit and Loss Statement')), ApiExperimentResultPageWrapper(cha
rts=[ApiExperimentResultGraphChartWrapper(chart=ApiChartMetadataShort(id=180288,
is_group_mode_enabled=False, layout=ApiChartLayoutData(height=1, start_col=0, sta
rt_row=1, width=2), name='ELT Service Level by Products (Per Product)', rc_id=180
254, type='LINE')), class_name='ApiExperimentResultGraphChartWrapper', data=ApiGra
phChartData(entity_list=[], id=180288, show_x=True, total_entity_count=0, total_t
echnical_row_count=0, type='LINE')), ApiExperimentResultGraphChartWrapper(chart=A

```

```

piChartMetadataShort(id=180289, is_group_mode_enabled=False, layout=ApiChartLayoutData(height=1, start_col=2, start_row=1, width=2), name='ELT Service Level by Products (Per Source)', rc_id=180254, type='LINE'), class_name='ApiExperimentResultGraphChartWrapper', data=ApiGraphChartData(entity_list=[], id=180289, show_x=True, total_entity_count=0, total_technical_row_count=0, type='LINE')), ApiExperimentResultGraphChartWrapper(chart=ApiChartMetadataShort(id=180290, is_group_mode_enabled=False, layout=ApiChartLayoutData(height=1, start_col=4, start_row=1, width=2), name='ELT Service Level by Revenue (Per Object)', rc_id=180254, type='LINE'), class_name='ApiExperimentResultGraphChartWrapper', data=ApiGraphChartData(entity_list=[], id=180290, show_x=True, total_entity_count=0, total_technical_row_count=0, type='LINE')), ApiExperimentResultGraphChartWrapper(chart=ApiChartMetadataShort(id=180291, is_group_mode_enabled=False, layout=ApiChartLayoutData(height=1, start_col=0, start_row=0, width=2), name='Service Level by Products (Per Product)', rc_id=180254, type='LINE'), class_name='ApiExperimentResultGraphChartWrapper', data=ApiGraphChartData(entity_list=[], id=180291, show_x=True, total_entity_count=0, total_technical_row_count=0, type='LINE')), ApiExperimentResultGraphChartWrapper(chart=ApiChartMetadataShort(id=180292, is_group_mode_enabled=False, layout=ApiChartLayoutData(height=1, start_col=2, start_row=0, width=2), name='Service Level by Products (Per Source)', rc_id=180254, type='LINE'), class_name='ApiExperimentResultGraphChartWrapper', data=ApiGraphChartData(entity_list=[], id=180292, show_x=True, total_entity_count=0, total_technical_row_count=0, type='LINE')), ApiExperimentResultGraphChartWrapper(chart=ApiChartMetadataShort(id=180293, is_group_mode_enabled=False, layout=ApiChartLayoutData(height=1, start_col=4, start_row=0, width=2), name='Service Level by Revenue (Per Object)', rc_id=180254, type='LINE'), class_name='ApiExperimentResultGraphChartWrapper', data=ApiGraphChartData(entity_list=[], id=180293, show_x=True, total_entity_count=0, total_technical_row_count=0, type='LINE'))], page=ApiPage(id='180287', name='Service Level')), ApiExperimentResultPageWrapper(charts=[ApiExperimentResultGraphChartWrapper(chart=ApiChartMetadataShort(id=180295, is_group_mode_enabled=False, layout=ApiChartLayoutData(height=1, start_col=3, start_row=0, width=3), name='Customer Tariffs', rc_id=180254, type='BAR'), class_name='ApiExperimentResultGraphChartWrapper', data=ApiGraphChartData(entity_list=[], id=180295, show_x=True, total_entity_count=0, total_technical_row_count=0, type='BAR')), ApiExperimentResultTableChartWrapper(chart=ApiChartMetadataShort(id=180296, is_group_mode_enabled=False, layout=ApiChartLayoutData(height=1, start_col=0, start_row=0, width=3), name='Customer Tariffs, Profit, Total Cost, Revenue', rc_id=180254, type='TABLE'), class_name='ApiExperimentResultTableChartWrapper', data=ApiStrictTableChartData(entity_list=[], id=180296, total_entity_count=0, total_technical_row_count=0, type='TABLE'))], page=ApiPage(id='180294', name='Tariffs'))))

```

```

In [16]: for p in the_run_results.pages:
          print(f"\n{p.page.name} (id:{p.page.id})")
          for c in p.charts:
              print(f"    - {c.chart.name} (type: {c.chart.type}) (id: {c.chart.id})")

```

Available Inventory (id:180256)

- Available Inventory (type: LINE) (id: 180257)
- Available Inventory Including Backlog (type: LINE) (id: 180258)
- Average Daily Available Inventory (type: LINE) (id: 180259)
- Average Daily On-hand Inventory (type: LINE) (id: 180260)
- On-hand Inventory (type: LINE) (id: 180261)

Fulfillment (id:180262)

- Demand Placed, Fulfillment Received (Accumulative) (type: TABLE) (id: 180263)
- Demand Placed, Fulfillment Received (Accumulative, Per Object) (type: TABLE) (id: 180264)
- Demand Placed, Fulfillment Received (Daily) (type: LINE) (id: 180265)
- Demand Placed, Fulfillment Received (Daily, Per Object) (type: LINE) (id: 180266)
- Demand Received, Fulfillment Shipped (Accumulative) (type: TABLE) (id: 180267)
- Demand Received, Fulfillment Shipped (Daily) (type: LINE) (id: 180268)

Lead Time (id:180269)

- Lead Time (type: LINE) (id: 180270)
- Max Lead Time (type: TABLE) (id: 180271)
- Mean Lead Time (Best-Mean-Worst) (type: BMW-LINE) (id: 180272)

Production (id:180273)

- Available Inventory Including Backlog (type: LINE) (id: 180274)
- Products Produced (Units) Factory Shenzhen (type: BAR) (id: 180277)
- Products Produced (Units) Factory Xiamen (type: BAR) (id: 180280)

Profit and Loss Statement (id:180283)

- Profit and Loss Statement (type: TABLE) (id: 180284)
- Profit, Revenue, Total Cost (type: BAR) (id: 180285)
- Revenue, Total Cost (type: LINE) (id: 180286)

Service Level (id:180287)

- ELT Service Level by Products (Per Product) (type: LINE) (id: 180288)
- ELT Service Level by Products (Per Source) (type: LINE) (id: 180289)
- ELT Service Level by Revenue (Per Object) (type: LINE) (id: 180290)
- Service Level by Products (Per Product) (type: LINE) (id: 180291)
- Service Level by Products (Per Source) (type: LINE) (id: 180292)
- Service Level by Revenue (Per Object) (type: LINE) (id: 180293)

Tariffs (id:180294)

- Customer Tariffs (type: BAR) (id: 180295)
- Customer Tariffs, Profit, Total Cost, Revenue (type: TABLE) (id: 180296)

Obtain the pages in the experiment's result dashboard

```
In [17]: try:
# Returns statistics pages for the result of experiment run.
# Exception fixed for 'id' attribute
# We use the experiment_result_id from the previously executed synchronous r
result_id = the_sync_result.experiment_result_id
dashboard_pages = api_instance.get_experiment_dashboard_pages(result_id)
print("The response of OpenApi->get_experiment_dashboard_pages:\n")

# CHANGED Displaying the available pages and charts with their dynamic IDs
for page in dashboard_pages:
    print(f"Dashboard Page: {page.name} (ID: {page.id})")
    for chart in page.charts:
```

```
print(f" - Chart: {chart.name} (ID: {chart.id})")

except Exception as e:
    print("Exception when calling OpenApiApi->get_experiment_dashboard_pages: %s"
```

The response of OpenApiApi->get_experiment_dashboard_pages:

Dashboard Page: Available Inventory (ID: 272062)

- Chart: Available Inventory (ID: 272063)
- Chart: Available Inventory Including Backlog (ID: 272064)
- Chart: Average Daily Available Inventory (ID: 272065)
- Chart: Average Daily On-hand Inventory (ID: 272066)
- Chart: On-hand Inventory (ID: 272067)

Dashboard Page: Fulfillment (ID: 272068)

- Chart: Demand Placed, Fulfillment Received (Accumulative) (ID: 272069)
- Chart: Demand Placed, Fulfillment Received (Accumulative, Per Object) (ID: 272070)
- Chart: Demand Placed, Fulfillment Received (Daily) (ID: 272071)
- Chart: Demand Placed, Fulfillment Received (Daily, Per Object) (ID: 272072)
- Chart: Demand Received, Fulfillment Shipped (Accumulative) (ID: 272073)
- Chart: Demand Received, Fulfillment Shipped (Daily) (ID: 272074)

Dashboard Page: Lead Time (ID: 272075)

- Chart: Lead Time (ID: 272076)
- Chart: Max Lead Time (ID: 272077)
- Chart: Mean Lead Time (Best-Mean-Worst) (ID: 272078)

Dashboard Page: Production (ID: 272079)

- Chart: Available Inventory Including Backlog (ID: 272080)
- Chart: Products Produced (Units) Factory Shenzhen (ID: 272083)
- Chart: Products Produced (Units) Factory Xiamen (ID: 272086)

Dashboard Page: Profit and Loss Statement (ID: 272089)

- Chart: Profit and Loss Statement (ID: 272090)
- Chart: Profit, Revenue, Total Cost (ID: 272091)
- Chart: Revenue, Total Cost (ID: 272092)

Dashboard Page: Service Level (ID: 272093)

- Chart: ELT Service Level by Products (Per Product) (ID: 272094)
- Chart: ELT Service Level by Products (Per Source) (ID: 272095)
- Chart: ELT Service Level by Revenue (Per Object) (ID: 272096)
- Chart: Service Level by Products (Per Product) (ID: 272097)
- Chart: Service Level by Products (Per Source) (ID: 272098)
- Chart: Service Level by Revenue (Per Object) (ID: 272099)

Dashboard Page: Tariffs (ID: 272100)

- Chart: Customer Tariffs (ID: 272101)
- Chart: Customer Tariffs, Profit, Total Cost, Revenue (ID: 272102)

Export the results as an excel matrix

```
In [18]: # Exception fixed for wrong IDs
# EXPORT ALL DASHBOARD PAGES TO EXCEL
try:
    # Dynamically select the first dashboard page to export to avoid hardcoded ID
    if dashboard_pages and len(dashboard_pages) > 0:
        target_result_id = the_sync_result.experiment_result_id
        print(f"Exporting ALL dashboard pages from result ID {target_result_id}.

        # Iterate over every page available in the dashboard
        for page in dashboard_pages:

            # Clean the page name to make it a safe Windows filename (replace sp
            safe_page_name = re.sub(r'[^a-zA-Z0-9_\-]', '_', page.name)
```

```

# Save the exported Excel file locally
# Create a unique filename for each page
output_filename = f"Simulation_Results_{safe_page_name}.xlsx"
output_path = os.path.join(os.getcwd(), output_filename)

print(f"Downloading '{page.name}' (ID: {page.id}) -> {output_filename}")

# Returns an Excel representation of the dashboard page
# Export this specific page
excel_export = api_instance.export_dashboard_page(target_result_id,

# Check if the API returns a temporary file path
# Save the file securely
if isinstance(excel_export, str) and os.path.exists(excel_export):
    shutil.move(excel_export, output_path)
elif isinstance(excel_export, bytes):
    with open(output_path, "wb") as file:
        file.write(excel_export)
else:
    print(f"Unexpected data format received for page {page.name}")

print("\n[SUCCESS] All dashboard pages exported successfully.")
else:
    print("No dashboard pages available to export.")

except Exception as e:
    print("Exception when calling OpenApiApi->export_dashboard_page: %s\n" % e)

```

Exporting ALL dashboard pages from result ID 272060...

```

Downloading 'Available Inventory' (ID: 272062) -> Simulation_Results_Available_In
ventory.xlsx
Downloading 'Fulfillment' (ID: 272068) -> Simulation_Results_Fulfillment.xlsx
Downloading 'Lead Time' (ID: 272075) -> Simulation_Results_Lead_Time.xlsx
Downloading 'Production' (ID: 272079) -> Simulation_Results_Production.xlsx
Downloading 'Profit and Loss Statement' (ID: 272089) -> Simulation_Results_Profit
_and_Loss_Statement.xlsx
Downloading 'Service Level' (ID: 272093) -> Simulation_Results_Service_Level.xlsx
Downloading 'Tariffs' (ID: 272100) -> Simulation_Results_Tariffs.xlsx

```

[SUCCESS] All dashboard pages exported successfully.

Close current project

```

In [19]: #Error fixed of indentation
try:
    # Closes the currently open project.
    api_response = api_instance.close_project()
    print("The response of OpenApiApi->close_project:\n")
    pprint(api_response)
    print("\nProject closed successfully.")
except ApiException as e:
    print("Exception when calling OpenApiApi->close_project: %s\n" % e)

```


The response of OpenApiApi->close_project:

```
ApiProjectResponse(accessible=True, creation_date='2026-03-10T16:19:55.145394Z',
current_user_id=None, deleted=False, id=79020, name='TFG_ADRIAN_ENCABO', owner_us
er_id=3)
```

Project closed successfully.

=====

PHASE 2 Flat AI Decision & Scenario Cloning

◀ ————— ▶

Install the dependencies (All required for PHASE 2)

```
In [20]: # Install xlwings to interact natively with Microsoft Excel
!{sys.executable} -m pip install xlwings
```

Requirement already satisfied: xlwings in c:\users\luis\downloads\tfg\anaconda\envs\tfg_anylogistix\lib\site-packages (0.35.1)

Requirement already satisfied: pywin32>=224 in c:\users\luis\downloads\tfg\anaconda\envs\tfg_anylogistix\lib\site-packages (from xlwings) (311)

```
In [ ]: import os
import shutil
import xlwings as xw #For excel modifications
import time
from pprint import pprint

import re # Used for cleaning text strings
```

1. Define the Flat AI logic (3 predefined decisions)

```
In [26]: decision_options = [
    "DECISION 0: Increase Demand by 20% (Aggressive Growth)",
    "DECISION 1: Decrease Transport Costs by 15% (Cost Optimization)",
    "DECISION 2: Increase Safety Stock by 10% (Conservative Risk Management)"
]

print("--- FLAT AI: EVALUATING PREDEFINED DECISIONS ---")
for i, decision in enumerate(decision_options):
    print(f"[{i}] {decision}")

# For this 'flat AI' phase, we will simulate the AI choosing a decision automati
# Let us assume the AI evaluates the current context and selects Decision 0.
selected_decision_index = 2

selected_decision_text = decision_options[selected_decision_index]

print(f"\nFLAT AI SELECTED: {selected_decision_text}")
```


--- FLAT AI: EVALUATING PREDEFINED DECISIONS ---

[0] DECISION 0: Increase Demand by 20% (Aggressive Growth)

[1] DECISION 1: Decrease Transport Costs by 15% (Cost Optimization)

[2] DECISION 2: Increase Safety Stock by 10% (Conservative Risk Management)

FLAT AI SELECTED: DECISION 2: Increase Safety Stock by 10% (Conservative Risk Management)

2. Flat AI Autonomous Data Modification

```
In [27]: # Define the paths for the original and the new modified scenario
ORIGINAL_EXCEL_FILENAME = "Global Network Examination.xlsx"
ORIGINAL_EXCEL_PATH = os.path.abspath(ORIGINAL_EXCEL_FILENAME)
MODIFIED_EXCEL_PATH = os.path.abspath(f"Modified_{ORIGINAL_EXCEL_FILENAME}")

print(f"Reading original scenario from: {ORIGINAL_EXCEL_PATH}")

# 1. Create a fresh copy of the original Excel to avoid corrupting the base one
try:
    shutil.copy2(ORIGINAL_EXCEL_PATH, MODIFIED_EXCEL_PATH)
    print("Created a clean copy of the scenario.")
except Exception as e:
    print(f"Error copying file: {e}")

print("Opening Microsoft Excel in the background to preserve strict formatting..")

# 2. Open Microsoft Excel silently (visible=False) so it doesn't interrupt you
app = xw.App(visible=False)

try:
    wb = app.books.open(MODIFIED_EXCEL_PATH)

    if selected_decision_index == 0:
        # DECISION 0: Increase Demand by 20%
        print(f"Applying {decision_options[selected_decision_index]}...")
        ws = wb.sheets['Demand']

        # Get the range of used columns and rows
        last_col = ws.used_range.last_cell.column
        last_row = ws.used_range.last_cell.row

        # Extract headers to find the right columns dynamically
        headers_raw = ws.range((1, 1), (1, last_col)).value
        headers = headers_raw if isinstance(headers_raw, list) else [headers_raw]

        if 'Col 7' in headers and 'Col 10' in headers:
            col7_idx = headers.index('Col 7') + 1
            col10_idx = headers.index('Col 10') + 1

            for r in range(2, last_row + 1):
                if ws.range((r, col7_idx)).value == 'Quantity':
                    current_val = ws.range((r, col10_idx)).value
                    if isinstance(current_val, (int, float)):
                        ws.range((r, col10_idx)).value = current_val * 1.20
            print("Demand increased by 20% successfully.")

        elif selected_decision_index == 1:
            # DECISION 1: Decrease Transport Costs by 15%
            print(f"Applying {decision_options[selected_decision_index]}...")
```

```

ws = wb.sheets['Paths']

last_col = ws.used_range.last_cell.column
last_row = ws.used_range.last_cell.row

headers_row = ws.range((1, 1), (1, last_col)).value
headers = headers_row if isinstance(headers_row, list) else [headers_row]

if 'Col 3' in headers and 'Col 4' in headers:
    col3_idx = headers.index('Col 3') + 1
    col4_idx = headers.index('Col 4') + 1

    for r in range(2, last_row + 1):
        val3 = ws.range((r, col3_idx)).value
        if val3 in ['Cost per unit', 'Cost']:
            current_val = ws.range((r, col4_idx)).value
            if isinstance(current_val, (int, float)):
                ws.range((r, col4_idx)).value = current_val * 0.85
            print("Transport costs decreased by 15% successfully.")

elif selected_decision_index == 2:
    # DECISION 2: Increase Safety Stock by 10%
    print(f"Applying {decision_options[selected_decision_index]}...")
    ws = wb.sheets['Inventory']

    last_col = ws.used_range.last_cell.column
    last_row = ws.used_range.last_cell.row

    headers_row = ws.range((1, 1), (1, last_col)).value
    headers = headers_row if isinstance(headers_row, list) else [headers_row]

    if 'Col 5' in headers and 'Col 6' in headers:
        col5_idx = headers.index('Col 5') + 1
        col6_idx = headers.index('Col 6') + 1

        for r in range(2, last_row + 1):
            if ws.range((r, col5_idx)).value == 'Safety stock':
                current_val = ws.range((r, col6_idx)).value
                if isinstance(current_val, (int, float)):
                    ws.range((r, col6_idx)).value = current_val * 1.10
            print("Safety stock increased by 10% successfully.")

# 3. Save changes explicitly using native Microsoft Excel
wb.save()
print(f"\nModified scenario safely saved to: {MODIFIED_EXCEL_PATH}")
print("Ready for ALX API Import.")

except Exception as e:
    print(f"An error occurred during native Excel modification: {e}")
finally:
    # 4. Always ensure the hidden Excel application is closed, even if it crashes
    try:
        wb.close()
    except:
        pass
    app.quit()

```

Reading original scenario from: C:\Users\Luis\Downloads\TFG\API\Global Network Examination.xlsx

Created a clean copy of the scenario.

Opening Microsoft Excel in the background to preserve strict formatting...

Applying DECISION 2: Increase Safety Stock by 10% (Conservative Risk Management)...

Safety stock increased by 10% successfully.

Modified scenario safely saved to: C:\Users\Luis\Downloads\TFG\API\Modified_Global Network Examination.xlsx

Ready for ALX API Import.

3. Modification by Excel

```
In [28]: # Create a unique name for the new scenario based on the AI decision
new_scenario_name = f"Modified_AI_Decision_{selected_decision_index}"
print(f"Importing '{MODIFIED_EXCEL_PATH}' as new scenario: '{new_scenario_name}'")

try:
    # We use import_excel to upload the file and create a new scenario.
    # Note: In most Python OpenAPI clients, passing the file path as a string is
    import_response = api_instance.import_excel(
        new_scenario_name=new_scenario_name,
        project_id=the_project.id,
        file=MODIFIED_EXCEL_PATH,
        need_to_import_experiments=True # We want to keep the experiments config
    )

    print("Import process initiated successfully.")
    pprint(import_response)

    # Since import_excel is asynchronous, we need to track the job_id until it f
    if import_response.job_id:
        print(f"\nTracking import job ID: {import_response.job_id}...")

        while True:
            # Get the current status of the import job
            import_status = api_instance.get_import_status(import_response.job_id)
            print(f"Current import status: {import_status.status}")

            # Check if the process has reached a final state
            if import_status.status in ['DONE', 'FAILED', 'CANCELED']:
                print(f"Import process finished with status: {import_status.stat

                if import_status.status == 'DONE':
                    # We save the ID of the newly created scenario for the next
                    new_scenario_id = import_status.scenario_id
                    print(f"\nSUCCESS! New scenario created with ID: {new_scenar

                else:
                    # In case it fails, it usually provides an error message
                    print(f"Error details (if any): {getattr(import_status, 'err

                break

            # Wait for 2 seconds before checking again
            time.sleep(2)
        else:
            print("Warning: No job_id received from the import response.")
```

```
except Exception as e:
    print(f"Exception when calling OpenApiApi->import_excel: {e}")
```

Importing 'C:\Users\Luis\Downloads\TFG\API\Modified_Global Network Examination.xlsx' as new scenario: 'Modified_AI_Decision_2'...

Import process initiated successfully.

ApiImportResponse(error=None, job_id=120)

Tracking import job ID: 120...

Current import status: DONE

Import process finished with status: DONE

SUCCESS! New scenario created with ID: 275557

4. Run simulation on AI scenario

```
In [31]: # Run simulation on the new AI scenario
# The AI automatically configures a SIMULATION experiment for its newly created
# and runs it synchronously to generate the results.

print(f"--- PHASE 2 - BLOCK 4: RUNNING SIMULATION FOR NEW AI SCENARIO (ID: {new_

try:
    print("1. Fetching SIMULATION configuration for the imported scenario...")
    # Retrieve experiments explicitly for the AI scenario
    ai_run_configurations = api_instance.get_experiments(the_project.id, new_sce

    # Select the 'SIMULATION' type experiment
    ai_simulation_RC = next((r for r in ai_run_configurations if r.type == 'SIMU

    if ai_simulation_RC:
        print(f"Found SIMULATION experiment (ID: {ai_simulation_RC.id}). Launchi

        # 2. Run the simulation
        sim_result = api_instance.run_experiment_synchronously(ai_simulation_RC.
        # Safely extract the result ID
        the_ai_sim_result_id = getattr(sim_result, 'experiment_result_id', None)

        if the_ai_sim_result_id:
            print(f"\n[SYSTEM] Simulation finished! Captured Result ID: {the_ai_

            # 3. Fetch the full list of dashboard pages (KPIs) and store them fo
            print("\n3. Fetching dashboard pages layout...")
            ai_dashboard_pages = api_instance.get_experiment_dashboard_pages(the
            print(f"Found {len(ai_dashboard_pages)} dashboard pages ready for ex

        else:
            print("Failed to capture 'experiment_result_id' from the synchronous
            pprint(sim_result)
    else:
        print("No SIMULATION experiment found for the AI scenario.")

except Exception as e:
    print("Exception during Simulation Execution: %s\n" % e)
```

--- PHASE 2 - BLOCK 4: RUNNING SIMULATION FOR NEW AI SCENARIO (ID: 275557) ---

1. Fetching SIMULATION configuration for the imported scenario...

Found SIMULATION experiment (ID: 276244). Launching synchronously...

[SYSTEM] Simulation finished! Captured Result ID: 276773

3. Fetching dashboard pages layout...

Found 7 dashboard pages ready for export.

5. Export the new dashboard pages

```
In [32]: # Export results as an Excel matrix
# The AI Loops through every available dashboard page (Finance, Inventory, etc.)
# and saves each one as a separate Excel file to preserve visual charts.

print(f"--- PHASE 2 - BLOCK 5: EXPORTING AI SIMULATION RESULTS ---")

try:
    if 'ai_dashboard_pages' in locals() and len(ai_dashboard_pages) > 0:
        print(f"Exporting ALL dashboard pages from AI result ID {the_ai_sim_resu

        # Iterate over every page available in the dashboard
        for page in ai_dashboard_pages:
            # Clean the page name to make it a safe Windows filename (replace sp
            safe_page_name = re.sub(r'^a-zA-Z0-9_\-', '_', page.name)

            # Use the selected_decision_index to name the file accordingly
            # Create a unique filename for each page, identifying it's from the
            output_filename = f"AI_Decision{selected_decision_index}_Results_{sa
            output_path = os.path.join(os.getcwd(), output_filename)

            print(f"Downloading '{page.name}' (ID: {page.id}) -> {output_filenam

            # Export this specific page
            excel_export = api_instance.export_dashboard_page(the_ai_sim_result_

            # Save the file securely (handle both string paths and byte streams)
            if isinstance(excel_export, str) and os.path.exists(excel_export):
                shutil.move(excel_export, output_path)
            elif isinstance(excel_export, bytes):
                with open(output_path, "wb") as file:
                    file.write(excel_export)
            else:
                print(f"Unexpected data format received for page {page.name}")

            print("\n[TOTAL SUCCESS] All AI dashboard pages exported successfully.")
        else:
            print("No dashboard pages available to export.")

    except Exception as e:
        print("Exception when exporting AI dashboard pages: %s\n" % e)
```

--- PHASE 2 - BLOCK 5: EXPORTING AI SIMULATION RESULTS ---

Exporting ALL dashboard pages from AI result ID 276773...

Downloading 'Available Inventory' (ID: 276775) -> AI_Decision2_Results_Available_Inventory.xlsx

Downloading 'Fulfillment' (ID: 276781) -> AI_Decision2_Results_Fulfillment.xlsx

Downloading 'Lead Time' (ID: 276788) -> AI_Decision2_Results_Lead_Time.xlsx

Downloading 'Production' (ID: 276792) -> AI_Decision2_Results_Production.xlsx

Downloading 'Profit and Loss Statement' (ID: 276802) -> AI_Decision2_Results_Profit_and_Loss_Statement.xlsx

Downloading 'Service Level' (ID: 276806) -> AI_Decision2_Results_Service_Level.xlsx

Downloading 'Tariffs' (ID: 276813) -> AI_Decision2_Results_Tariffs.xlsx

[TOTAL SUCCESS] All AI dashboard pages exported successfully.

=====

DEVELOPMENT TESTS - IGNORE THEM

◀ ————— ▶

```
In [ ]: #Avoid self-signed warnings
import os
import urllib3
```

For self-signed certificates it is necessary to download them and add to the certification path

```
In [ ]: import ssl
import socket

def download_self_signed_cert_no_validation(hostname, port, cert_file):
    context = ssl._create_unverified_context()
    conn = context.wrap_socket(socket.socket(socket.AF_INET), server_hostname=hostname)
    conn.connect((hostname, port))
    der_cert = conn.getpeercert(binary_form=True)
    pem_cert = ssl.DER_cert_to_PEM_cert(der_cert)

    with open(cert_file, 'w') as cert_file:
        cert_file.write(pem_cert)

    print(f"Certificate saved to {cert_file.name}")

# Example usage
download_self_signed_cert_no_validation(SERVER_IP, 443, MY_CERT_FILENAME)
```

```
In [ ]: # !/usr/bin/openssl s_client -showcerts -connect {SERVER_IP}:443 </dev/null 2>/d
```

```
In [ ]: MY_CERT_FILE=os.path.abspath(MY_CERT_FILENAME)
if os.path.isfile(MY_CERT_FILE):
    print(f"Cert file OK: {MY_CERT_FILE} ")
else:
    print(f"Cert file ERROR: {MY_CERT_FILE} ")
MY_CERT_FILE
```

```
In [ ]: os.environ['REQUESTS_CA_BUNDLE'] = MY_CERT_FILE

# Create a PoolManager with a custom CA bundle
http = urllib3.PoolManager(
    cert_reqs='CERT_NONE',      # Enforce cert verification
    ca_certs=MY_CERT_FILE       # Path to your self-signed cert
)

urllib3.disable_warnings(urllib3.exceptions.InsecureRequestWarning)
http.request("GET", TEST_URL)
```

```
In [ ]: from urllib3 import HTTPSConnectionPool

pool = HTTPSConnectionPool(
    SERVER_IP,
    port=443,
    cert_reqs='CERT_REQUIRED',
    ca_certs=MY_CERT_FILE
)
```

```
In [ ]: import requests

# Any request you make now will use your certificate
response = requests.get(TEST_URL, verify=False)
print(response.text)
```